

High-Performance WS2812B USB Streaming Guide with Complete Sender and Receiver Code

Overview

This document provides a complete, production-ready implementation for streaming LED data from a Windows desktop application using Cinder to a Raspberry Pi Pico 2 W running MicroPython over USB Serial, driving WS2812B LEDs efficiently and reliably.

Binary Data Format

Each LED uses 3 bytes: R, G, B. For 240 LEDs, each frame is exactly 720 bytes. Frames are sent continuously with no headers or parsing required.

Complete Cinder Sender Example (C++)

```
#include "cinder/Serial.h"
#include "cinder/Log.h"
#include <vector>

using namespace ci;

class LedSender {
public:

    static const int NUM_LEDS = 240;
    static const int FRAME_SIZE = NUM_LEDS * 3;

    Serial serial;
    std::vector<uint8_t> buffer;

    LedSender(const std::string& portName)
    {
        serial = Serial(portName, 2000000);
        buffer.resize(FRAME_SIZE);
    }

    void sendFrame(const std::vector<Color8u>& leds)
    {
        for(int i = 0; i < NUM_LEDS; i++)
        {
            buffer[i*3+0] = leds[i].r;
            buffer[i*3+1] = leds[i].g;
            buffer[i*3+2] = leds[i].b;
        }

        serial.writeBytes(buffer.data(), FRAME_SIZE);
    }
};
```

MicroPython boot.py Configuration

```
import usb_cdc
usb_cdc.enable(console=False, data=True)
```

Complete Optimized MicroPython Receiver (main.py)

```

import usb_cdc
import machine
import neopixel
import gc

NUM_LEDS = 240
FRAME_SIZE = NUM_LEDS * 3
LED_PIN = 0

gc.disable()

cdc = usb_cdc.data

frame_buffer = bytearray(FRAME_SIZE)
frame_view = memoryview(frame_buffer)

pin = machine.Pin(LED_PIN, machine.Pin.OUT)
np = neopixel.NeoPixel(pin, NUM_LEDS)

np_buf = np.buf

def read_frame():
    read = 0
    while read < FRAME_SIZE:
        n = cdc.readinto(frame_view[read:])
        if n:
            read += n

while True:

    read_frame()

    np_buf[:] = frame_buffer

    np.write()

```

Performance Summary

With 240 WS2812B LEDs, maximum achievable frame rate is approximately 120–135 FPS due to physical LED protocol timing. For installation use cases, 30–60 FPS provides excellent performance and visual smoothness.